

Competitive Programming

Section 39: Heaps and Priority Queues

Competitive Programming Course

2026-07-10

Table of contents

1	Priority queues as frontier managers	3
2	Two heaps for medians	4
3	Sweep with ending times	5
4	K-way merging	6
5	Python and C++ heap details	7
6	Checklist	8
7	Local exercises	9
8	How to use the checker	10
9	Worked decision traces	11
9.1	Running median: preserve two halves	11
9.2	Course rooms: a heap is the active frontier	11
9.3	K smallest pair sums: merge sorted rows	12
10	Implementation and proof habits	13
11	Failure modes, exercises, and workflow	14

1 Priority queues as frontier managers

A heap maintains the smallest or largest currently available item. It is the right tool when the next action is determined by one key, but future insertions can change the frontier.

You should be able to say what one heap entry means. Examples:

<code>(value)</code>	<code>current candidate number</code>
<code>(end_time)</code>	<code>room that becomes free at this time</code>
<code>(sum, i, j)</code>	<code>pair $A[i]+B[j]$ from stream i at position j</code>
<code>(distance, vertex)</code>	<code>frontier state in Dijkstra</code>

The heap does not prove the algorithm by itself. The proof comes from showing that the smallest or largest key is safe to process next.

2 Two heaps for medians

Keep the lower half in a max-heap and the upper half in a min-heap:

```
max(low) <= min(high)
len(low) == len(high) or len(low) == len(high)+1
```

Then the lower median is `max(low)`.

Example insertion sequence:

```
insert 5: low=[5], high=[]      median=5
insert 1: low=[1], high=[5]     median=1
insert 9: low=[5,1], high=[9]   median=5
insert 2: low=[2,1], high=[5,9] median=2
```

After every insertion:

1. Put the value in the correct side.
2. Swap tops if order is violated.
3. Rebalance sizes.

3 Sweep with ending times

To count rooms for intervals, process intervals by start time. A min-heap stores end times of currently occupied rooms. Before a new interval starts, pop all rooms whose end time is at most the start. The heap size is the number of simultaneous intervals.

For half-open intervals $[l, r)$, an interval ending at time t does not overlap an interval starting at t . Therefore pop while:

```
end_time <= next_start
```

For closed intervals $[l, r]$, the condition would be different.

4 K-way merging

If arrays A and B are sorted, each fixed i gives a sorted stream:

```
A[i]+B[0], A[i]+B[1], A[i]+B[2], ...
```

Put the first item of every stream in a heap. Repeatedly pop the smallest sum and push the next sum from the same stream. This is the same idea as merging many sorted lists.

Worked example:

```
A = [1, 4]
B = [2, 3, 10]
```

Streams:

```
i=0: 3, 4, 11
i=1: 6, 7, 14
```

Start heap:

```
(3,0,0), (6,1,0)
```

Pop (3,0,0), then push (4,0,1). This produces sorted pair sums without generating every pair in advance.

5 Python and C++ heap details

Python's `heapq` is a min-heap. For a max-heap, store negative values:

```
import heapq
low = []
heapq.heappush(low, -x)
largest = -low[0]
```

C++ `priority_queue<int>` is a max-heap by default. For a min-heap:

```
priority_queue<int, vector<int>, greater<int>> pq;
```

For pair sums, store all fields needed to generate the next candidate:

```
(current_sum, i, j)
```

If you store only `current_sum`, you cannot know which next pair to push.

6 Checklist

Define exactly what one heap entry represents.
Make sure stale entries are removed or ignored.
For two heaps, rebalance after every insertion.
For interval problems, decide whether touching endpoints overlap.

7 Local exercises

- A. Running Median: After every inserted number, print the lower median of the prefix.
- B. Course Rooms: Given half-open intervals $[l, r)$, print the minimum number of rooms needed.
- C. K Smallest Pair Sums: Two arrays are sorted. Print the k smallest values of $a[i] + b[j]$.

8 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/39_heaps_priority_queues/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.
Exercises

9 Worked decision traces

9.1 Running median: preserve two halves

After every insertion, **low** is a max-heap holding the smaller half and **high** is a min-heap holding the larger half. Maintain both facts:

```
every value in low <= every value in high
size(low) == size(high) or size(low) == size(high)+1
```

For prefix 5, 1, 9, 2, the lower medians are traced below.

insert	low (max heap)	high (min heap)	printed
5	{5}	{}	5
1	{1,5} -> {1}	{5}	1
9	{1}	{5,9} -> move 5	5
2	{1,2}	{5,9}	2

The arrows are the rebalance step. Once the two facts hold, exactly $\text{ceil}(\text{prefix_length}/2)$ values are at most **low.top()**, so it is the requested lower median. Each push/pop costs $O(\log n)$.

9.2 Course rooms: a heap is the active frontier

Sort half-open intervals by start. The min-heap contains end times of courses that have started but not yet ended. For (0,10), (5,7), (6,20), (10,12):

start	interval	remove ends <= start	active end heap after push
0	[0,10)	none	{10}
5	[5,7)	none	{7,10}
6	[6,20)	none	{7,10,20}
10	[10,12)	remove 7,10	{12,20}

The maximum heap size is three. The $\leq$ is essential: half-open intervals ending at time 10 and starting at time 10 do not overlap.

9.3 K smallest pair sums: merge sorted rows

If both arrays are sorted, each fixed i creates a sorted row $a[i]+b[0], a[i]+b[1], \dots$. Seed the heap with each row's first value and advance only the row that was popped. For $a=[1,4,8]$, $b=[2,3,10]$:

```
initial heap: (3,row0,col0), (6,row1,col0), (10,row2,col0)
pop 3, push 4; pop 4, push 11; pop 6, push 7; pop 7, push 14
```

Thus the first four sums are 3,4,6,7. The heap always holds the smallest unreported element of every nonempty row, so its minimum is globally next. Only seed $\min(n,k)$ rows in a generalized variant; the local contract has $k \leq n \cdot m$.

10 Implementation and proof habits

C++. `priority_queue<T>` is a max-heap; write `priority_queue<T, vector<T>, greater<T>>` for a min-heap. Store pair sums in `long long`, and store `(sum,row,col)` as a tuple so lexicographic comparison breaks ties safely.

Python. `heapq` is a min-heap. Negate values for a max-heap, and use tuples such as `(sum,row,col)` rather than custom comparators. Never call `heappop` on an empty heap; prove the seed/advance invariant first.

11 Failure modes, exercises, and workflow

Problem	Wrong shortcut	Test
A	sort every prefix	descending input of length 2e5
B	remove ends < <code>start</code>	[0,10) and [10,12)
C	seed every pair	two arrays of length 2e5 with small <code>k</code>

1. Prove that moving `high.top()` to `low` repairs both running-median facts.
2. Explain why the room heap size after cleanup equals the number of overlapping intervals at the current start time.
3. On paper, list the first five sums for [0,7] and [1,2,9] using rows.

Answers: the moved element is the smallest large-half value; every remaining end is strictly after the current start; the sums are 1,2,8,9,9.

Run `CP_TARGET=solution python3 sections/39_heaps_priority_queues/check.py`. For A compare with a sorted prefix, for B brute-force overlap counts on small interval sets, and for C enumerate and sort every pair sum when `n,m <= 8`.

Use A to maintain the two heap size invariant, B to free rooms before allocating a new one, and C to make each heap item represent the next unseen pair. Test duplicate values and equal endpoints.